

Lab 03: Use an Internal Application Load Balancer

Google Cloud Associate Cloud Engineer | GSP041

Overview

Built a two-tier architecture using an Internal Application Load Balancer. The backend tier runs a private prime number calculator service across multiple VMs with no public internet access. The frontend tier is a public-facing web server that queries the internal service through the load balancer and displays results in a color-coded prime number matrix.

Architecture

 Public Internet	 Frontend Web Server	 Internal ALB + Backend VMs
<i>Users access frontend IP</i>	<i>Has public IP, queries internal ALB</i>	<i>No public IP — private only</i>

Objectives

- Create a Python-based prime number calculator backend
- Deploy backend VMs using a managed instance group
- Set up an internal Application Load Balancer
- Test the internal load balancer from inside the network
- Create a public-facing frontend that uses the internal service

Task 1 — Create Virtual Environment

Set up a Python virtual environment in Cloud Shell to isolate dependencies for this lab.

```
sudo apt-get install -y virtualenv
python3 -m venv venv
source venv/bin/activate
gcloud services enable clouddaicompanion.googleapis.com
```

```

Selecting previously unselected package python3-filelock.
Preparing to unpack .../1-python3-filelock_3.13.1-1_all.deb ...
Unpacking python3-filelock (3.13.1-1) ...
Selecting previously unselected package python3-platformdirs.
Preparing to unpack .../2-python3-platformdirs_4.2.0-1_all.deb ...
Unpacking python3-platformdirs (4.2.0-1) ...
Selecting previously unselected package python3-wheel-whl.
Preparing to unpack .../3-python3-wheel-whl_0.42.0-2_all.deb ...
Unpacking python3-wheel-whl (0.42.0-2) ...
Preparing to unpack .../4-python3-virtualenv_20.25.0+ds-2_all.deb ...
Unpacking python3-virtualenv (20.25.0+ds-2) ...
Selecting previously unselected package virtualenv.
Preparing to unpack .../5-virtualenv_20.25.0+ds-2_all.deb ...
Unpacking virtualenv (20.25.0+ds-2) ...
Setting up python3-filelock (3.13.1-1) ...
Setting up python3-distlib (0.3.6-1) ...
Setting up python3-platformdirs (4.2.0-1) ...
Setting up python3-wheel-whl (0.42.0-2) ...
Setting up python3-virtualenv (20.25.0+ds-2) ...
Setting up virtualenv (20.25.0+ds-2) ...
Processing triggers for man-db (2.12.0-4build2) ...
student_04_2cd9cf97d8f98cloudshell:~ (qwiklabs-gcp-04-dac122e6elf1)$ python3 -m venv venv
student_04_2cd9cf97d8f98cloudshell:~ (qwiklabs-gcp-04-dac122e6elf1)$ source venv/bin/activate
(venv) student_04_2cd9cf97d8f98cloudshell:~ (qwiklabs-gcp-04-dac122e6elf1)$ gcloud compute instance-templates create primecalc \
--metadata-from-file startup-script-backend.sh \
--no-address --tags backend --machine-type=e2-medium
Created [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-04-dac122e6elf1/global/instanceTemplates/primecalc].
NAME: primecalc
MACHINE_TYPE: e2-medium
PREEMPTIBLE:
CREATION_TIMESTAMP: 2026-05-17T04:51:39.976-07:00

```

Task 2 — Create Backend Managed Instance Group

Created a startup script (backend.sh) containing a Python web server that checks if a number is prime. Used Gemini Code Assist to explain the script. Then deployed it as a managed instance group of 3 VMs with no public IP addresses for security.

Instance template (no public IP for security):

```
gcloud compute instance-templates create primecalc --metadata-from-file startup-script=backend.sh --no-address --tags backend --machine-type=e2-medium
```

Firewall rule to allow port 80 to backend VMs:

```
gcloud compute firewall-rules create http --network default --allow=tcp:80 --source-ranges [IP] --target-tags backend
```

Create managed instance group with 3 VMs:

```
gcloud compute instance-groups managed create backend --size 3 --template primecalc --zone [ZONE]
```

```

--machine-type=e2-medium
To search the help text of gcloud commands, run:
gcloud help -- SEARCH_TERMS
(venv) student_04_2cd9cf97d8f9@cloudshell:~ (qwiklabs-gcp-04-dac122e6e1f1)$
gcloud compute firewall-rules create http --network default --allow=tcp:80 \
--source-ranges 10.150.0.0/20 --target-tags backend
Creating firewall...working..Created [https://www.googleapis.com/compute/v1/
/projects/qwiklabs-gcp-04-dac122e6e1f1/global/firewalls/http].
Creating firewall...done.
NAME: http
NETWORK: default
DIRECTION: INGRESS
PRIORITY: 1000
ALLOW: tcp:80
DENY:
(venv) student_04_2cd9cf97d8f9@cloudshell:~ (qwiklabs-gcp-04-dac122e6e1f1)$
gcloud compute instance-groups managed create backend \
--size 3 \
--template primecalc \
--zone us-east4-a
Created [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-04-dac
122e6e1f1/zones/us-east4-a/instanceGroupManagers/backend].
NAME: backend
LOCATION: us-east4-a
SCOPE: zone
BASE_INSTANCE_NAME: backend
SIZE: 0
TARGET_SIZE: 3
INSTANCE_TEMPLATE: primecalc
AUTOSCALED: no
(venv) student_04_2cd9cf97d8f9@cloudshell:~ (qwiklabs-gcp-04-dac122e6e1f1)$

```

The screenshot shows the Google Cloud Platform Compute Engine console. The 'VM instances' tab is selected, displaying a list of three instances: 'backend-6kh1', 'backend-rf36', and 'backend-s268'. All instances are in the 'us-east4' zone and have a status of 'Running' (indicated by a green checkmark). The 'Connect' column shows 'SSH' for each instance. Below the list, there are 'Related actions' and a 'Hide' button. The bottom of the console shows the project name 'qwiklabs-gcp-04-dac122e6e1f1' and an 'Editor' button.

Status	Name	Zone	Connect
☒	backend-6kh1	us-east4	SSH
☒	backend-rf36	us-east4	SSH
☒	backend-s268	us-east4	SSH

Task 3 — Set Up the Internal Load Balancer

The Internal ALB has 3 main components: a health check, a backend service, and a forwarding rule. Unlike the external ALBs in Labs 01 and 02, this one has no public IP — it is only reachable from inside the VPC network.

Health check (checks if /2 returns 200 OK):

```
gcloud compute health-checks create http ilb-health --request-path /2
```

Backend service (internal scheme):

```
gcloud compute backend-services create prime-service --load-balancing-scheme internal --region=[REGION] --protocol tcp --health-checks ilb-health
```

Add instance group to backend:

```
gcloud compute backend-services add-backend prime-service --instance-group backend --instance-group-zone=[ZONE] --region=[REGION]
```

Forwarding rule (private static IP):

```
gcloud compute forwarding-rules create prime-lb --load-balancing-scheme internal --ports 80 --network default --region=[REGION] --address [IP] --backend-service prime-service
```

☐	Name	Load balancer type	Access type	Protocols	Region	Backends	Actions
☐	prime-service	Network (Passthrough)	Internal	TCP	us-east4	⚠ 1 regional backend service (1 instance group, 0 network endpoint groups)	⋮

Task 4 — Test the Internal Load Balancer

Created a temporary test VM inside the same VPC to verify the internal ALB is working. From inside the test VM, sent curl requests to the internal load balancer IP to check prime numbers.

```
gcloud compute instances create testinstance --machine-type=e2-standard-2 --zone [ZONE]
gcloud compute ssh testinstance --zone [ZONE]
curl [IP]/2    # Expected: True
curl [IP]/4    # Expected: False
curl [IP]/5    # Expected: True
gcloud compute instances delete testinstance --zone=[ZONE]
```

```
student-02-a43535be170e@testinstance:~$ curl 10.142.0.10/2
Truestudent-02-a43535be170e@testinstance:~$ curl 10.142.0.10/4
Falsesstudent-02-a43535be170e@testinstance:~$ curl 10.142.0.10/5
Truesstudent-02-a43535be170e@testinstance:~$ exit
logout
Connection to 35.190.191.173 closed.
(venv) student_02_a43535be170e@cloudshell:~ (qwiklabs-gcp-03-46dbb258dee0) $
```

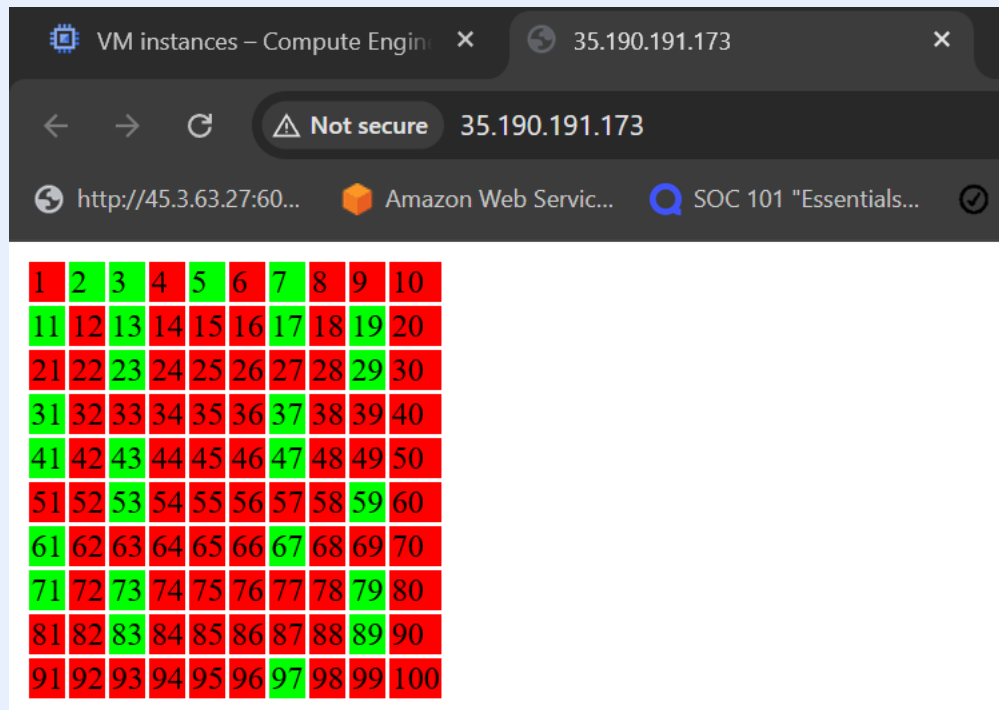
Task 5 — Create Public-Facing Frontend Web Server

Created a frontend VM with a public IP that runs a Python web server. The frontend queries the internal ALB to check if numbers are prime and displays a color-coded HTML matrix — green for prime, red for not prime.

```
gcloud compute instances create frontend --zone=[ZONE] --metadata-from-file
startup-script=frontend.sh --tags frontend --machine-type=e2-standard-2
gcloud compute firewall-rules create http2 --network default --allow=tcp:80 --
source-ranges 0.0.0.0/0 --target-tags frontend
```

The screenshot displays the Google Cloud Platform console interface for the Compute Engine service. The left-hand navigation pane is open, showing the 'Compute Engine' section with various options like Overview, Security risk overview, and Virtual machines. Under 'Virtual machines', 'VM instances' is selected. The main content area is titled 'VM instances' and includes tabs for 'Instances', 'Observability', and 'Instance schedules'. The 'Instances' tab is active, showing a table of VM instances. The table has columns for Status, Name, Zone, Recon, and Connect. Four instances are listed: 'backend-5wv0', 'backend-sqgh', 'backend-zxlj', and 'frontend'. All instances have a status of 'Running' (indicated by a green checkmark) and are located in the 'us-east1-c' zone. The 'frontend' instance is highlighted. Below the table, there is a 'Related actions' section. The bottom of the console shows a terminal window with the command 'gcloud compute instances create frontend --zone=[ZONE] --metadata-from-file startup-script=frontend.sh --tags frontend --machine-type=e2-standard-2' and a toolbar with icons for 'Open Editor' and other actions.

Status	Name	Zone	Recon	Connect
Running	backend-5wv0	us-east1-c		SSH
Running	backend-sqgh	us-east1-c		SSH
Running	backend-zxlj	us-east1-c		SSH
Running	frontend	us-east1-c		SSH



Real World Application

This two-tier architecture is used everywhere in enterprise applications:

- Banking — A public website talks to internal payment processing servers that are never exposed to the internet
- Healthcare — A patient portal queries internal medical record services privately
- E-commerce — A storefront queries internal inventory and pricing services through a private load balancer
- Microservices — Internal services talk to each other through internal load balancers without ever touching the public internet

What I Learned

- Internal ALBs have no public IP, they are only reachable from within the VPC
- Backend VMs with --no-address have no public internet access, making them more secure
- Managed instance groups automatically replace failed VMs for high availability
- Health checks ensure traffic only goes to VMs that are responding correctly
- A two-tier architecture separates public-facing and private internal services
- Gemini Code Assist can explain complex scripts directly in the Cloud Shell IDE

Challenges & How I Solved Them

- Forgot to SSH into the test instance before running curl commands.
Ran curl directly from Cloud Shell instead of from inside the test VM, which caused the session to immediately exit since the internal ALB is not reachable from outside the VPC network.
Fixed by running the SSH command first:
`gcloud compute ssh testinstance --zone [ZONE]`
then running curl from inside the instance.

Resources

GCP Internal ALB Docs: <https://cloud.google.com/load-balancing/docs/internal>
Lab ID: GSP041